Das Projekt *InventarWorkerService* für Auszubildende Fachinformatik der GWDG

Inhaltsverzeichnis

Teaser DE 2
Teaser EN 2
Einleitung 3
Teilprojekte 4
InventarWorkerService 4
ServiceStatusReaderApp 7
CtrlWorkerServiceApp 9
CtrlWorkerServiceCmdlet 12
InventarViewerApp 14
Hilfsmittel 17
C#/.Net Framework 17
JetBrains Rider 17
PowerShell 18
GitLab 21
SQLite-Datenbank 22
MongoDB-Datenbank 22
NuGet-Pakete 22
PS-Gallery 22
Erweiterungsmöglichkeiten 24
Ausblick 26
Links ..... **Fehler! Textmarke nicht definiert.**
Zusammenfassung 27
Summary 28
Anhang 29
Gemeni-cli 30
Claude-Code 34

Teaser DE

Beginnend mit dem Ausbildungsjahr ab August 2025 werden die Auszubildenden Fachinformatiker*in für Anwendungsentwicklung und Systemintegration nach vielen Jahren des Bestehens der beiden Ausbildungsgänge bei der GWDG in gemeinsamen Programmierübungen in den Anforderungen des Rahmenlehrplans für die ersten beiden Lehrjahre in diesen Inhalten unterrichtet.

Teaser EN

Starting with the training year from August 2025, the trainees IT specialist for application development and system integration will be instructed in the requirements of the framework curriculum for the first two years of the apprenticeship after many years of passing the two training courses at the GWDG in joint programming exercises.

Einleitung

Das Projekt, oder besser der Proof-of-Concept, kurz PoC, InventarWorkerService ist das neue Projekt für die gemeinsame Ausbildung der Fachinformatiker*innen in den Fachrichtungen Anwendungsentwicklung und Systemintegration bei der GWDG.

Das Projekt umfasst die Erstellung eines Systemdienstes, Kommandozeilen Programm(e), ein PowerShell Cmdlet, PowerShell-Skripte, Datenspeicherung in eine SQL-Datenbank und Übertragung von JSON-Daten per REST API. Da die Daten überwiegend im JSON-Format vorliegen, soll sich auch mit einer NoSQL-Datenbank beschäftigt werden.

Alle dafür benötigten Programme, Integrierte Entwicklungsumgebung (engl. **I**ntegrated **D**evelopment **E**nvironment, kurz IDE), Programmiersprache C# und das .Net-Framework von Microsoft, Microsoft PowerShell und die verwendeten (No)SQL-Datenbank(en) laufen auf allen drei Betriebssystemplattformen macOS, Linux und Windows.

Die eingesetzte Programmiersprache ist Microsoft C# und das dazugehörige .Net Framework. Die Entwicklung selbst findet in der C# IDE JetBrains Rider statt. So weit sind alle Teilprojekte eine Code-Basis, die aber nach der Übersetzung (engl. Compiling) auf allen drei genannten Betriebssystemen ausgeführt werden können.

Das Projekt umfasst mit seinem Umfang die meisten Teile des Rahmenausbildungsplan der Lehrjahre 1 und 2 und die darin geforderten gemeinsamen Inhalte für Fachinformatiker*innen für die Programmioerung.

Weiterhin können sich die Auszubildenden beider Fachrichtungen ergänzen. Die Anwendungsentwickler*innen kommen von der klassischen Anwendungsentwicklung, z.B. Desktop- oder Web-Apps, aber haben so tiefere Einblicke in Richtung Betriebssystem, Kommandozeile und Systemdienste. Systemintegrator*innen verfügen in der Regel über fundierte Kenntnisse im Bereich Betriebssysteme, Systemdienste und Kommandozeilenanwendungen und erweitern ihr Fachwissen durch Einblicke in die Anwendungsentwicklung. Somit können beide Fachrichtungen im Zusammenspiel voneinander und miteinander profitieren.

Das Projekt soll außerdem die Zusammenarbeit und den Austausch der Auszubildenden im Bereich Fachinformatik mit- und untereinander bei der GWDG fördern.

Teilprojekte

In den folgenden Abschnitten werden kurz die Teilprojektes des gesamten Projekts, bzw. PoC, kurz vorgestellt sowie die verwendeten Hilfsmittel, wie z.B. IDE, Datenbanken, Programmiersprache etc.. Neben der Beschreibung auch mit zusätzlichen Abbildungen und mit Teil-Quellzeilenauflistungen.

InventarWorkerService

Das Kernstück dieses Projekts. Ein Programm, das als Systemdienst auf allen drei Plattformen, Windows, macOS und Linux, registriert und ausgeführt werden kann.

Das Programm ist eine Quellzeilenbasis und kann nach der Übersetzung auf den genannten Plattformen bzw. Betriebssystemen ohne weitere Anpassungen laufen.

Das Programm bzw. Systemdienst liest die installierte Hard- und Software des aktuellen Systems aus. In den Quellzeilen sind „Weichen“ bzw. Abfragen, auf welchem Betriebssystem das Programm bzw. Dienst läuft und führt dann zum ermittelten Betriebssystem die entsprechenden betriebssystem-spezifischen Abfragen aus.

Die ermittelte Hard- und Software wird über eine REST API bereitgestellt und kann von jeglichem Programm angefragt werden, dass es versteht mit einer REST API zu interagieren. In diesem Beispiel-Projekt ist das die *InventarViewerApp*, die im weiteren Verlauf des Artikels noch kurz vorgestellt wird.

```
/// <summary>
/// Continuously executes the worker's main logic for processing inventory
tasks
/// and updates the status service with the current execution state.
/// </summary>
/// <param name="stoppingToken">A cancellation token that signals the
worker to stop.</param>
/// <returns>A Task representing the asynchronous execution of the worker's
main loop.</returns>
protected override async Task ExecuteAsync(CancellationToken stoppingToken)
{
    _statusService.WriteStatus(new ServiceStatus
    {
        State = "Starting",
        StartTime = _startTime,
        ProcessedItems = 0
    });

    while (!stoppingToken.IsCancellationRequested)
    {
        try
        {
            // Hardware- und Software-Inventarisierung durchführen
            await ProcessInventoryItems();

            _processedItems++;
```

```
            // Status aktualisieren
            _statusService.WriteStatus(new ServiceStatus
            {
                State = "Running",
                StartTime = _startTime,
                ProcessedItems = _processedItems,
                LastActivity = DateTime.Now
            });

            // Statistiken aktualisieren
            _statusService.WriteStatistics(new ServiceStatistics
            {
                TotalProcessedItems = _processedItems,
                AverageProcessingTime = CalculateAverageProcessingTime(),
                Uptime = DateTime.Now - _startTime,
                MemoryUsage = GC.GetTotalMemory(false)
            });

            var message = $"Inventarisierung abgeschlossen:
{_processedItems} Durchläufe";
            _logger.LogInformation(message);
            _statusService.WriteLog(message);

            await Task.Delay(30000, stoppingToken);
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, "Fehler in Worker");
            _statusService.WriteLog($"ERROR: {ex.Message}");
            _statusService.WriteStatus(new ServiceStatus
            {
                State = "Error",
                StartTime = _startTime,
                ProcessedItems = _processedItems,
                LastError = ex.Message
            });

            await Task.Delay(5000, stoppingToken);
        }
    }

    _statusService.WriteStatus(new ServiceStatus
    {
        State = "Stopped",
        StartTime = _startTime,
        ProcessedItems = _processedItems,
        LastActivity = DateTime.Now
    });
}
```

Quellzeilenausschnitt_1: Zeigt die Kernmethode des *InventarWorkerService*, der das aktuelle Hard- und Softwareinventar des Systems ermittelt, auf dem dieser ausgeführt wird. Weiterhin werden einfache statistische Daten geschrieben, die das Programm

ServiceStatusReaderApp ausliest, wenn dieses Programm auf dem gleichen System ausgeführt wird.

```
info: InventarWorkerService.Worker[0]
      Starte Hardware- und Software-Inventarisierung...
info: InventarWorkerService.Services.Hardware.HardwareInventoryService[0]
      Sammle Hardware-Informationen...
info: InventarWorkerService.Services.Software.SoftwareInventoryService[0]
      Sammle Software-Inventar...
info: InventarWorkerService.Services.Hardware.HardwareInventoryService[0]
      Hardware-Informationen erfolgreich gesammelt
info: InventarWorkerService.Services.Software.SoftwareInventoryService[0]
      Software-Inventar erfolgreich gesammelt
info: InventarWorkerService.Worker[0]
      Inventar gespeichert.
info: InventarWorkerService.Worker[0]
      === Inventar Zusammenfassung ===
      System: MacBook-Air-von-Thorsten (macOS)
      CPU: Unknown (8 Kerne)
      Speicher: 0 GB total, 0,0% verwendet
      Festplatten: 14 Laufwerke
      Netzwerk: 22 Schnittstellen
      Software: 128 installierte Programme
      Prozesse: 455 laufende Prozesse
      Services: 0 Windows-Services
      Uptime: 0 Tage, 00:00
      ==============================
info: InventarWorkerService.Worker[0]
      Inventarisierung erfolgreich abgeschlossen
info: InventarWorkerService.Worker[0]
      Inventarisierung abgeschlossen: 18 Durchläufe
```

Bildschirmausschnitt_1: Zeigt die Ausgaben zur Laufzeit des InventarWorkerService an und gibt Auskunft darüber, dass die Hard- und Software des Systems ermittelt worden ist. Diese Ausgaben sind sichtbar, wenn das Programm manuell zu Test-Zwecken gestartet wurde. Wenn es als Dienst vom Betriebssystem gestartet wurde, sollen diese Ausgaben in die zentrale Log-Datei des Betriebssystems geschrieben werden.

ServiceStatusReaderApp

Ein einfaches Kommandozeilen-Programm, dass auf dem aktuellen System ausgeführt werden kann, um zu ermitteln, ob der *InventarWorkerService* aktuell und wie viele Male gelaufen ist.

Das Programm läuft nur in einer Kommandozeile des jeweiligen Betriebssystems und gibt seine Informationen ebenfalls als Text auf der Kommandozeile aus.

```
/// <summary>
/// Reads the current service status from the specified status file.
/// </summary>
/// <returns>
/// An instance of <see
cref="InventarWorkerCommon.Models.Service.ServiceStatus"/>
/// containing the current status information, or null if the status file
does not exist
/// or if an error occurs during the reading process.
/// </returns>
public InventarWorkerCommon.Models.Service.ServiceStatus? ReadStatus()
{
    try
    {
        var statusFile = Path.Combine(_statusDirectory, "status.json");
        if (!File.Exists(statusFile)) return null;

        var json = File.ReadAllText(statusFile);
        return
JsonSerializer.Deserialize<InventarWorkerCommon.Models.Service.ServiceStatu
s>(json, _jsonOptions);
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Fehler beim Lesen des Status: {ex.Message}");
        return null;
    }
}
```

Quellzeilenausschnitt_2: Zeigt die Kernmethode des Programms *ServiceStatusReaderApp*, dass den Status des *InventarWorkerService* aus der Datei *status.json* ausliest, in die der *InventarWorkerService* zur Laufzeit seinen Status hineinschreibt

```
Run    InventarWorkerService ×    ServiceStatusReaderApp ×

/Users/thorstenhindermann/RiderProjects/InventarWorkerService/ServiceS
=== InventarWorkerService Status ===

Service läuft: JA
Zustand: Running
Gestartet: 2025-07-30 07:56:23
Letzte Aktivität: 2025-07-30 07:57:24
Verarbeitete Items: 3

=== Statistiken ===
Gesamt verarbeitete Items: 3
Durchschnittliche Verarbeitungszeit: 20185,17 ms
Laufzeit: 00.00:01:00
Speicherverbrauch: 14,00 MB

=== Letzte 10 Log-Einträge ===
2025-07-30 07:56:24 - Inventarisierung abgeschlossen: 1 Durchläufe
2025-07-30 07:56:54 - Inventarisierung abgeschlossen: 2 Durchläufe
2025-07-30 07:57:24 - Inventarisierung abgeschlossen: 3 Durchläufe

Process finished with exit code 0.
```

Bildschirmausschnitt_2: Das Programm *ServiceStatusReaderApp* ist ausgeführt woden und hat ermittelt, wieviele Male der *InventarWorkerService* gelaufen ist und gibt noch ein paar statistische Daten aus.

CtrlWorkerServiceApp

Ein Steuerungsprogramm, dass auf der Kommandozeile läuft und auf dem Betriebssystem, auf dem auch der *InventarWorkerService* läuft.

Das Programm kann mit den Parametern *Start* und *Stop* den Dienst entsprechend starten und stoppen. Über den Parameter *--tui* kann eine grafische Steuerung aufgerufen werden, die es dem Nutzenden erlaubt, mittels Tastatur und/oder Maus den Dienst über die entsprechenden Schaltflächen zu starten und stoppen. Besonderheit, es wird hier ASCII-Grafik genutzt, die auch nur in der Kommandozeile zur Verfügung steht.

```
/// <summary>
/// Starts the service for the specified operating system.
/// </summary>
/// <remarks>
/// This method checks the operating system at runtime and attempts to
/// start the service using platform-specific logic. Supported platforms
include
/// Windows, Linux, macOS, and FreeBSD. If the current operating system is
not
/// supported, a <see cref="PlatformNotSupportedException"/> will be
thrown.
/// </remarks>
/// <exception cref="PlatformNotSupportedException">
/// Thrown when the method is executed on an unsupported platform.
/// </exception>
public void StartService()
{
    if (RuntimeInformation.IsOSPlatform(OSPlatform.Windows))
    {
        StartWindowsService();
    }
    else if (RuntimeInformation.IsOSPlatform(OSPlatform.Linux))
    {
        StartLinuxService();
    }
    else if (RuntimeInformation.IsOSPlatform(OSPlatform.OSX))
    {
        StartMacOSService();
    }
    else if (RuntimeInformation.IsOSPlatform(OSPlatform.FreeBSD))
    {
        StartFreeBSDService();
    }
    else
    {
        throw new PlatformNotSupportedException("Plattform wird nicht
unterstützt");
    }
}
/// <summary>
/// Starts the service for the Linux operating system.
```

```
/// </summary>
/// <remarks>
/// This method executes the Linux system command to start the service
/// associated with the specified service name. It utilizes the `systemctl`
/// command to initiate the service. The service name is converted to
lowercase
/// to ensure compatibility with Linux's case-sensitive file and service
naming system.
/// </remarks>
/// <exception cref="InvalidOperationException">
/// Thrown when the execution of the `systemctl` command fails, or if an
error occurs
/// during the command execution process.
/// </exception>
private void StartLinuxService()
{
    ExecuteCommand("systemctl", $"start {_serviceName.ToLower()}.service");
}
/// <summary>
/// Executes a system command with the specified filename and arguments.
/// </summary>
/// <param name="fileName">The name of the executable file or command to
run.</param>
/// <param name="arguments">The arguments to pass to the command.</param>
/// <exception cref="InvalidOperationException">
/// Thrown when the command execution fails or if the process exits with a
non-zero code.
/// </exception>
private void ExecuteCommand(string fileName, string arguments)
{
    try
    {
        using var process = Process.Start(new ProcessStartInfo
        {
            FileName = fileName,
            Arguments = arguments,
            UseShellExecute = false,
            RedirectStandardOutput = true,
            RedirectStandardError = true
        });

        process?.WaitForExit();

        if (process?.ExitCode != 0)
        {
            var error = process.StandardError.ReadToEnd();
            throw new InvalidOperationException($"Befehl fehlgeschlagen:
{error}");
        }
    }
    catch (Exception ex)
    {
        throw new InvalidOperationException($"Fehler beim Ausführen von
'{fileName} {arguments}': {ex.Message}");
```

```
    }
}
```

Quellzeilenausschnitt_3: Zeigt zuerst die Kernmethode die ermittelt, auf welchem Betriebssystem der Programm *CtrlWorkerServiceApp* ausgeführt wird. Die nächsten beiden Methoden zeigen, wie der als Betriebssystem-Dienst registrierte InventarWorkerService auf dem Betriebssystem Linux gestartet wird.

Bildschirmausschnitt_3: Zeigt den Start des Dienstes mittels des Programm *CtrlWorkerServiceApps* und die Ausgabe und anschließende Überprüfung, ob der Dienst läuft, unter dem Betriebssystem macOS.

CtrlWorkerServiceCmdlet

Die gleiche Funktionalität *CtrlWorkerServiceApp*, nur es sich hierbei um ein Microsoft PowerShell `Cmdlet` handelt, dass eben nur in der PowerShell Kommandozeile ausgeführt werden kann. Aber auch die PowerShell steht für alle Plattformen zur Verfügung.

```
/// <summary>
/// Cmdlet to provide control actions for a Worker Service.
/// Allows starting, stopping, or launching a terminal user interface (TUI)
for the service.
/// </summary>
/// <remarks>
/// The cmdlet supports multiple parameter sets for different operations.
/// By default, it acts on a service with the name 'mein-service', but this
can be overridden using the ServiceName parameter.
/// Errors encountered during the process are captured and written using
the WriteError method.
/// </remarks>
[Cmdlet(VerbsLifecycle.Invoke, "WorkerServiceControl")]
[OutputType(typeof(string))]
public class InvokeWorkerServiceControlCmdlet : PSCmdlet
{
    [Parameter(
        Mandatory = false,
        Position = 0,
        ParameterSetName = "Start",
        HelpMessage = "Startet den Worker Service")]
    public SwitchParameter Start { get; set; }

    [Parameter(
        Mandatory = false,
        Position = 0,
        ParameterSetName = "Stop",
        HelpMessage = "Stoppt den Worker Service")]
    public SwitchParameter Stop { get; set; }

    [Parameter(
        Mandatory = false,
        Position = 0,
        ParameterSetName = "TUI",
        HelpMessage = "Startet die Terminal-Benutzeroberfläche")]
    public SwitchParameter Tui { get; set; }

    [Parameter(
        Mandatory = false,
        HelpMessage = "Name des Services (Standard: 'mein-service')")]
    [ValidateNotNullOrEmpty]
    public string ServiceName { get; set; } = "mein-service";
```

Quellzeilenausschnitt_4: Zeigt die zur Verfügungstellung des PowerShell Cmdlet-Namens und der Cmdlet-Parameter und deren Zuordnung den jeweiligen ParameterSets im PowerShell Cmdlet *CtrlWorkerServiceCmdlet*.

```
net9.0 pwsh
PowerShell 7.5.2
net9.0 Import-Module ./CtrlWorkerServiceCmdlet.dll
net9.0 Get-Module

ModuleType Version    PreRelease Name                                ExportedCommands
---------- -------    ---------- ----                                ----------------
Binary     1.0.0.0               CtrlWorkerServiceCmdlet             Invoke-WorkerServiceControl
Manifest   7.0.0.0               Microsoft.PowerShell.Management     {Add-Content, Clear-Content, Clear-It…
Manifest   7.0.0.0               Microsoft.PowerShell.Utility        {Add-Member, Add-Type, Clear-Variable…
Script     0.0                   oh-my-posh-core                     {Enable-PoshLineError, Enable-PoshToo…
Script     2.3.6                 PSReadLine                          {Get-PSReadLineKeyHandler, Get-PSRead…

thorstenhindermann net9.0 main ≡ Invoke-WorkerServiceControl -
Start               Verbose             InformationAction    InformationVariable
Stop                Debug               ProgressAction       OutVariable
Tui                 ErrorAction         ErrorVariable        OutBuffer
ServiceName         WarningAction       WarningVariable      PipelineVariable
net9.0 Invoke-WorkerServiceControl -Start -ServiceName inventarworkerservice -Verbose
VERBOSE: Starte Service 'inventarworkerservice'...
Service 'inventarworkerservice' wird gestartet...
VERBOSE: Service 'inventarworkerservice' erfolgreich gestartet.
thorstenhindermann net9.0 main ≡                                   in pwsh at 12:25:33
```

Bildschirmausschnitt_4: Zeigt das Laden, Überprüfen, Auflistung des Befehls und Aufruf des PowerShell Cmdlets *CtrlWorkerServiceCmdlet*.

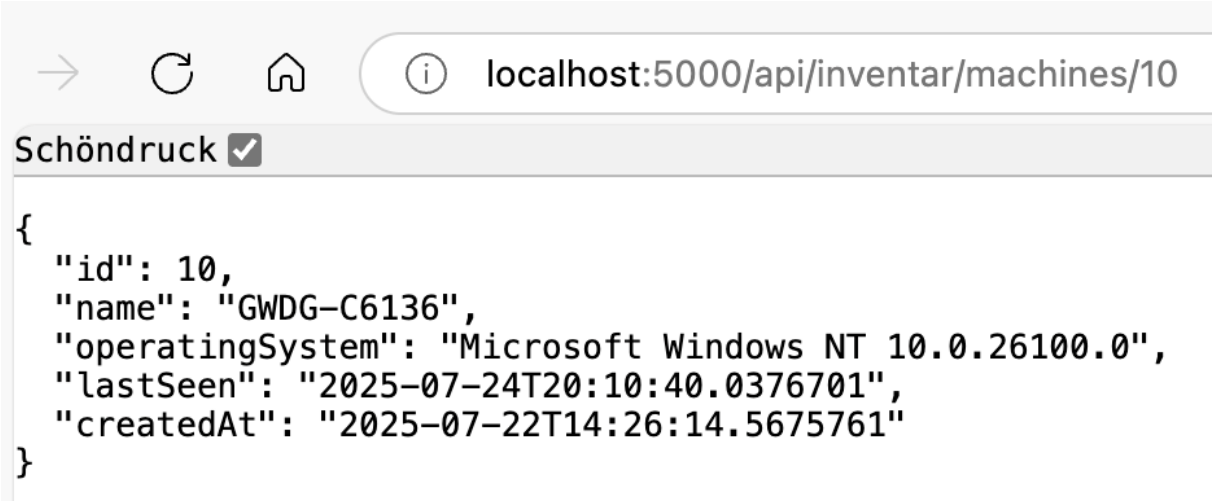
InventarViewerApp

Die *InventarViewerApp* stellt eine Anwendung zur Visualisierung und Verwaltung von IT-Inventardaten dar. Entwickelt in C# mit .NET 9.0, kombiniert sie zwei wesentliche Nutzungskonzepte: eine interaktive Terminal-Benutzeroberfläche für den direkten Zugriff und eine REST-basierte Web-API für die programmatische Integration externer Systeme. Die Softwarearchitektur der InventarViewerApp folgt einem klaren modularen Aufbau, dessen Aufbau und Beschreibung den Rahmen dieses Artikels sprengen würde.

```
/// <summary>
/// Controller that provides endpoints related to inventory management,
focusing on machines
/// and their associated hardware and software details.
/// </summary>
[ApiController]
[Route("api/[controller]")]
public class InventarController : ControllerBase
{
    private readonly DatabaseService _databaseService;

    public InventarController(DatabaseService databaseService)
    {
        _databaseService = databaseService;
    }

    /// <summary>
    /// Retrieves a list of all machines from the database, including their
details such as ID,
    /// name, operating system, and timestamps for creation or last seen
activity.
    /// </summary>
    /// <returns>A task that represents the asynchronous operation. The
task result contains
    /// an <see cref="ActionResult{T}"/> containing a list of <see
cref="Machine"/> objects,
    /// or an error response in case of a failure.</returns>
    [HttpGet("machines")]
    public async Task<ActionResult<List<Machine>>> GetMachines()
    {
        try
        {
            var machines = await _databaseService.GetMachinesAsync();
            return Ok(machines);
        }
        catch (Exception ex)
        {
            return StatusCode(500, new { message = "Fehler beim Abrufen der
Maschinen", error = ex.Message });
        }
    }

    /// <summary>
```

```
    /// Retrieves a specific machine by its unique numeric identifier from
the database.
    /// </summary>
    /// <param name="id">The unique identifier of the machine to
retrieve.</param>
    /// <returns>A task that represents the asynchronous operation. The
task result contains
    /// an <see cref="ActionResult{T}"/> containing the <see
cref="Machine"/> object if found,
    /// or an error response if the machine does not exist or an unexpected
error occurs.</returns>
    [HttpGet("machines/{id:int}")]
    public async Task<ActionResult<Machine>> GetMachine(int id)
    {
        try
        {
            var machine = await _databaseService.GetMachineByIdAsync(id);
            if (machine == null)
                return NotFound(new { message = "Maschine nicht gefunden"
});

            return Ok(machine);
        }
        catch (Exception ex)
        {
            return StatusCode(500, new { message = "Fehler beim Abrufen der
Maschine", error = ex.Message });
        }
    }
```

Quellzeilenausschnitt_5: Zeigt die Abfrage der SQL-Datenbanktabelle mit den ermittelten Maschinendaten von verschiedenen Rechner-Systemen. Einmal die Abfrage und Rückgabe aller Maschinen in der zugehörigen Datenbanktabelle und einmal nur die angaben eines spezifischen Rechner-Systems anhand der eindeutigen `Id` in der Datenbanktabelle. Die Ausgabe dieser Daten erfolgt als JSON-Ausgabe am jeweilig definierten REST API Endpunkt, der als `HttpGet`-Attribut der Methode vorangestellt ist.



Bildschirmausschnitt_5: Zeigt die Abfrage der REST API der Maschinen-Tabelle mit einer bestimmten Maschinen-`Id`. Die Ausgabe im Web Browser Edge wird mit einer lesbareren Formatierung, erreicht durch das Setzen des Hakens bei „Schöndruck“, angezeigt.

Hilfsmittel

In den kommenden Abschnitten werden kurz die verwendeten Hilfsmittel vorgestellt, die in diesem Beispiel-Projekt genutzt werden.

C#/.Net Framework

Die für dieses Projekt und dessen Teilprojekte verwendete Programmiersprache ist C# und das Rahmenwerk, sprichwörtlich kann man auch Werkzeugkasten sagen, ist das .Net Framework von Microsoft.

C# ist eine vielseitige Programmiersprache, die von Microsoft entwickelt wurde und sich durch ihre Einfachheit und Leistungsfähigkeit auszeichnet. Sie wird häufig für die Entwicklung von Desktop-Anwendungen, Webanwendungen, mobilen Apps und Spielen verwendet. C# bietet eine klare und strukturierte Syntax, die sowohl für Anfänger als auch für erfahrene Entwickler leicht verständlich ist.

Das .NET Framework ist ein umfassendes sogenanntes Fullstack-Framework, das eine breite Palette von Tools und Bibliotheken für die Entwicklung von Anwendungen jeglicher Art bereitstellt. Es unterstützt verschiedene Programmiersprachen, darunter C#, und ermöglicht die Erstellung von Anwendungen für verschiedene Plattformen wie Windows, macOS und Linux. Mit dem .NET Framework können Entwickler sowohl die Frontend- als auch die Backend-Komponenten ihrer Anwendungen effizient entwickeln und verwalten. Es bietet eine robuste Laufzeitumgebung und umfangreiche Klassenbibliotheken, die die Produktivität und Qualität der Softwareentwicklung erheblich steigern.

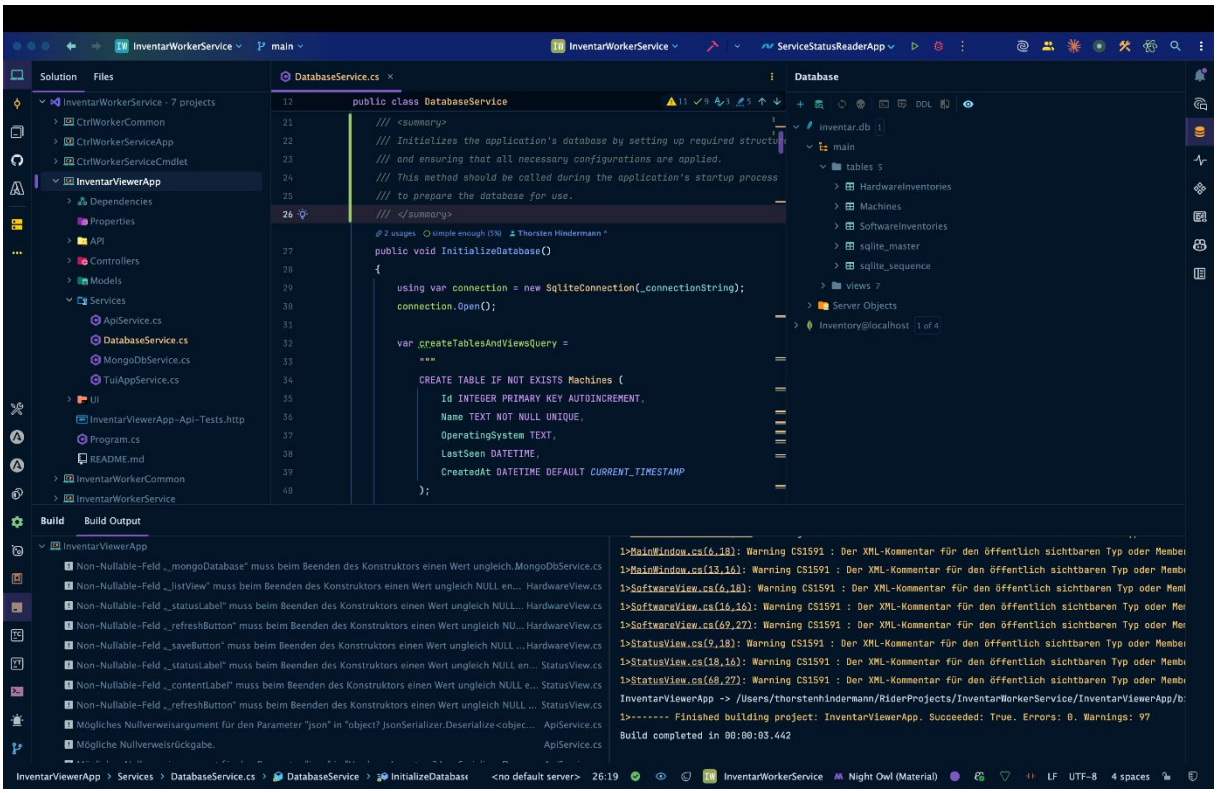
JetBrains Rider

Eine integrierte Entwicklungsumgebung, engl. Integrated Development Environment, kurz IDE, mit der das Projekt und dessen Teilprojekte erstellt werden. Mit Hilfe dieses Werkzeugs können die C#-Quellzeilen-Dateien erstellt werden, die übersetzt zu ausführbaren Programmen oder Bibliotheken werden. Weiterhin können mit diesem Werkzeug auch die verwendeten Datenbanken und deren Daten erstellt, untersucht und bearbeitet werden. Die Programme und Bibliotheken können auf Fehler untersucht werden und diese bei Entdeckung bereinigt werden. Soweit kann alles Notwendige mit diesem Werkzeug erledigt werden, was den integrierten Charakter diese Werkzeugs unterstreicht und sich ja auch in diesem damit erstellten Beispiel-Projekt wiederspiegelt.

Ein Vorteil von JetBrains Rider, wie von allen JetBrains IDEs, ist, dass diese auf allen drei hauptsächlichen Betriebssystemen, Windows, macOS und Linux, ausgeführt werden können und überall gleich aussehen und gleichermaßen bedient werden können. Weiterhin ist JetBrains Rider die auf C#/.Net Framework spezialisierte IDE von JetBrains und ist genau das richtige Werkzeug der Wahl um diese Aufgabe umzusetzen.

Das ist deswegen nein Vorteil, weil alle Auszubadenden Rechner mit den genannten Betriebssystemen haben.

Aus diesem Grund ist es wichtig einen gemeinsamen Nenner zu finden, um diese Aufgabe mit den Auszubildenden möglich zu machen. Und mit der Kombination von JetBrains Rider und dem C#/.Net Framework kann dies alles realisiert werden.



Bildschirmausschnitt_6: Zeigt die IDE in Aktion. Mit Hilfe dieses Werkzeugs kann unter anderem an den Quellzeilen gearbeitet werden, Datenbanken angezeigt und untersucht werden, die erstellten Quellzeilen übersetzt werden und eine übersichtliche Navigation durch die Teilprojekte des gesamten Projekts ist möglich. Es gibt aber noch viel mehr Hilfestellungen in der IDE, diese aber alle vorzustellen würde den Rahmen dieses Artikels bei weitem sprengen.

PowerShell

Die Microsoft PowerShell ist ein plattformübergreifende Automatisierungs- und Konfigurationsmanagement-Framework. Sie besteht aus einer Befehlszeilenschnittstelle und einer Skriptsprache, die speziell für die Systemadministration und Automatisierung von Aufgaben entwickelt wurde.

PowerShell ermöglicht es im Normalfall Fachinformatiker*innen Systemintegration, vereinfacht als Administrator*innen bezeichnet, Aufgaben zu automatisieren, indem sie Skripte schreiben, die eine Vielzahl von Systemverwaltungsaufgaben ausführen können. Diese Aufgaben können von der Verwaltung von Betriebssystemen und Anwendungen bis hin zur Automatisierung von Netzwerk- und Cloud-Diensten reichen.

Einige der Hauptmerkmale von PowerShell sind:

- **Cmdlets**: Dies sind spezialisierte .NET-Klassen, die eine bestimmte Funktionalität bereitstellen. Sie können verwendet werden, um Aufgaben wie das Verwalten von Dateien, das Ändern von Systemeinstellungen und das Verwalten von Prozessen auszuführen.
- **Skripte**: PowerShell-Skripte sind Textdateien, die eine Reihe von PowerShell-Befehlen enthalten. Diese Skripte können verwendet werden, um komplexe Aufgaben zu automatisieren und wiederholbare Prozesse zu erstellen.

```
<#
    .SYNOPSIS
       EntryPoint

    .DESCRIPTION
       Entry point function of the script

    .PARAMETER Start
       Switch for starting the WorkerService

    .PARAMETER Stop
       Stop the WorkerService

    .PARAMETER Tui
       Show Terminal GUI for controlling the WorkerService

    .PARAMETER Help
       Show usage of the controlling WorkerService script

    .PARAMETER ServiceName
       Override the default service name

    .EXAMPLE
       PS C:\> .\CtrlWorkerService.ps1

    .NOTES
       Additional information about the file.
#>
[CmdletBinding(DefaultParameterSetName = 'Tui')]
param
(
    [Parameter(ParameterSetName = 'Start',
             Mandatory = $false,
             Position = 0,
             HelpMessage = 'Switch for starting the WorkerService')]
    [switch]$Start = $false,
    [Parameter(ParameterSetName = 'Stop',
             Mandatory = $false,
             Position = 1,
             HelpMessage = 'Stop the WorkerService')]
    [switch]$Stop = $false,
    [Parameter(ParameterSetName = 'Tui',
             Mandatory = $false,
             Position = 2,
```

```
                HelpMessage = 'Show Terminal GUI for controlling the
WorkerService')]
    [switch]$Tui = $false,
    [Parameter(ParameterSetName = 'Help',
                Mandatory = $false,
                Position = 3,
                HelpMessage = 'Show usage of the controlling
WorkerService script')]
    [switch]$Help = $false,
    [Parameter(Mandatory = $false,
                Position = 4,
                HelpMessage = 'Override the default service name')]
    [ValidateNotNullOrEmpty()]
    [string]$ServiceName = 'default-service'
)
function Invoke-WorkerServiceControl
{
    try
    {
        if ($Start)
        {
            Start-WorkerService -ServiceName $ServiceName
        }
        elseif ($Stop)
        {
            Stop-WorkerService -ServiceName $ServiceName
        }
        elseif ($Tui)
        {
            Start-TerminalGui -ServiceName $ServiceName
        }
        else
        {
            Write-Host "Service Controller PowerShell Script - Hilfe" -
ForegroundColor Cyan
            Write-Host "=============================================="
-ForegroundColor Cyan
            Write-Host ""
            Write-Host "Verwendung:"
            Write-Host "  Invoke-WorkerServiceControl -Start [-
ServiceName <Name>]"
            Write-Host "  Invoke-WorkerServiceControl -Stop [-
ServiceName <Name>]"
            Write-Host "  Invoke-WorkerServiceControl -Tui [-
ServiceName <Name>]"
            Write-Host ""
            Write-Host "Parameter:"
            Write-Host "  -Start         Startet den angegebenen
Service"
            Write-Host "  -Stop          Stoppt den angegebenen
Service"
            Write-Host "  -Tui           Startet die Terminal-
Benutzeroberfläche"
            Write-Host "  -ServiceName   Name des Services (Standard:
```

```
'mein-service')"
            Write-Host ""
            Write-Host "Beispiele:"
            Write-Host "  Invoke-WorkerServiceControl -Start"
            Write-Host "  Invoke-WorkerServiceControl -Stop -
ServiceName 'custom-service'"
            Write-Host "  Invoke-WorkerServiceControl -Tui"
        }
    }
    catch
    {
        Write-Error -Message $_.Exception.Message -Category
OperationStopped
    }
}
```

Quellzeilenausschnitt_6: Zeigt PowerShell Script, das ähnlich dem Cmdlet aufgebaut ist um den angegebenen Dienst zu starten und zu stoppen. Eine Verwendungshilfe ist ebenfalls abrufbar.

- **Pipelines**: PowerShell ermöglicht es Benutzern, die Ausgabe eines Befehls als Eingabe für einen anderen Befehl zu verwenden. Dies ermöglicht es, komplexe Aufgaben durch das Verketten einfacher Befehle zu erstellen.
- **Objektorientierung**: Im Gegensatz zu vielen anderen Skriptsprachen arbeitet PowerShell mit Objekten anstelle von Text. Dies bedeutet, dass die Ausgabe eines Befehls ein Objekt ist, das Eigenschaften und Methoden hat, die von anderen Befehlen verwendet werden können.
- **PowerShell-Provider**: Dies sind spezialisierte PowerShell-Bibliotheken, die es ermöglichen, verschiedene Datenspeicher und Dienste so zu behandeln, als wären sie Dateisysteme. Dies bedeutet, dass Nutzende mit diesen Datenspeichern und Diensten auf die gleiche Weise interagieren können, wie sie es mit Dateien und Ordnern auf einem Dateisystem tun würden. PowerShell-Provider bieten eine einheitliche Schnittstelle für den Zugriff auf verschiedene Arten von Daten, wie z.B. das Dateisystem, die Registrierung, Zertifikatspeicher und mehr. Dies erleichtert die Verwaltung und Automatisierung von Aufgaben, da Administratoren und Entwickler dieselben Befehle und Techniken verwenden können, unabhängig davon, mit welchem Datenspeicher oder Dienst sie arbeiten.
  Aus der PS-Gallery können weitere PowerShell-Provider geladen und genutzt werden, unter anderem zum Beispiel für SQLite-Datenbanken.

GitLab

Bei der GWDG selbst betriebene Server für das Versions -Kontrollprogramm `git`. Die Firma Gitlab hat eine ausgefeilte Weboberfläche über das von Linus Torwalds entwickelte `git` erstellt, das neben den klassischen Versionskontrolle noch viele weitere Funktionalität bereitstellt. Diese erweiterten Funktionalitäten sind erst einmal

nicht Bestandteil dieses Projekts, sondern zuerst nur die Versionskontrolle der erstellten Quellzeilendateien.

SQLite-Datenbank

SQLite ist eine Programm-Bibliothek, die eine volle Implementierung des SQL-Standards beinhaltet, aber keine Installation eines SQL-Server erfordert. Alle Daten werden über eine Programmierschnittstelle und entsprechende SQL-Befehle in die Datenbank geschrieben, geändert, gelesen oder gelöscht.

MongoDB-Datenbank

MongoDB ist ein leistungsfähiges, dokumentenorientiertes Datenbankmanagementsystem, das sich durch seine hohe Flexibilität und Skalierbarkeit auszeichnet. Im Gegensatz zu traditionellen, relationalen Datenbanken speichert MongoDB Daten in JSON-ähnlichen Dokumenten, was eine dynamische und schemalose Datenstruktur ermöglicht. Dies macht es besonders geeignet für Anwendungen, die große Mengen an unstrukturierten oder sich schnell ändernden Daten verarbeiten müssen. MongoDB bietet zudem weitere Funktionen wie z.B. Replikation und eine leistungsstarke Abfragesprache, die Entwicklern hilft, komplexe Datenabfragen effizient zu gestalten und zu verwalten.

MongoDB lässt sich nahtlos in C#-Programme integrieren, indem man das offizielle MongoDB .NET Treiberpaket verwendet. Dieses Paket bietet eine umfangreiche API, die es Entwicklern ermöglicht, CRUD-Operationen (Erstellen, Lesen, Aktualisieren, Löschen) auf MongoDB-Datenbanken durchzuführen. Mit dem Treiber können Entwickler MongoDB-Datenbanken einfach verbinden, Abfragen erstellen und Daten manipulieren. Die Integration von MongoDB in C#-Anwendungen bietet eine flexible und skalierbare Lösung für die Verwaltung von Daten, insbesondere wenn es um große Mengen an unstrukturierten oder sich schnell ändernden Daten geht.

NuGet-Pakete

NuGet-Pakete sind Programm-Bibliotheken, die spezialisierte Funktionen gegenüber der bereits umfangreichen Basis des .NET Frameworks zur Verfügung stellen. Diese Pakete werden in verschiedenen Teilprojekten genutzt, um spezifische Anforderungen zu erfüllen.

PS-Gallery

Die PS-Gallery ist ein Repository für PowerShell-Module, Skripte und Ressourcen. Sie ermöglicht es Administrator*inne, PowerShell-Module und -Skripte zu durchsuchen, herunterzuladen und zu installieren, die von der PowerShell-Community erstellt und geteilt wurden.

Die PS-Gallery bietet eine zentrale Anlaufstelle für die Verwaltung und Verteilung von PowerShell-Inhalten, was die Automatisierung und Verwaltung von Systemen

erleichtert. Benutzer können ihre eigenen Module und Skripte hochladen und teilen, um anderen in der Community zu helfen und von deren Feedback zu profitieren.

Erweiterungsmöglichkeiten

Die InventarViewApp kann um die Möglichkeit erweitert werden, dass Systeme registriert werden können, deren Inventar gesammelt werden soll. Dazu müsste passenderweise die Tabelle `Machines` mit den Maschinen-Daten um z.B. Spalten für die IP-Adresse, den DNS-Namen erweitert werden. Weiterhin können Spalten dazukommen welche Maschine nicht mehr beim Sammeln der Daten beachtet werden soll (deaktiviert) oder gar aus dem Bestand genommen worden ist (deprovisioniert). Hier müsste man sich überlegen, was mit den bis dahin gesammelten Daten passiert (löschen, exportieren und/oder archivieren).

Weiterhin kann die REST API erweitert werden, damit in den REAT API Endpunkte anstatt der Maschinenname die Maschinen-ID aus der Datenbank als Referenzierung der Datensätze verwendet wird.

Weiterhin müsste es auf dem Rechner mit der InventarViewerApp einen weiteren WorkerSerivce geben, der die Tabelle `Machines` periodisch ausliest und die darin (nicht deaktivierten oder -provisionierten) Maschinen nach und nach deren Daten abfragt. Hier kommen dann Fragestellungen auf, was mit aktiven Maschinen passiert, die aber nicht kontinuierlich abgefragt werden können, z.B. Laptop auf Dienstreise oder lange Zeit ausgeschaltete Rechner (z.B. Urlaubszeit). Für den ersten Fall müsste der lokal auf den Rechnern vom *InventarWorkerService* gesammelten Daten auf die Festplatte des Laptops schreiben – was der InventarWorkerService aber schon tut. Hier müssten dann die Delta-Daten an Hard- und Software von den lokal gespeicherten Dateien zum Sammler-Service übertragen werden. Welche Daten fehlen, könnte anhand des letzten gespeicherten Datums in der Spalte `LastSeen` verglichen mit dem aktuellen Datum erreicht werden.

Derzeit werden die großen Datenbestände der ermittelten Software, laufenden Dienste etc. im JSON-Format in die SQL-Datenbank gespeichert. Hier könnten Überlegungen angestellt werden, dass für solche Datenbestände eine NoSQL-Datenbank wie z.B. MongoDB besonders gut geeignet ist, die JSON-Daten darin zu speichern und daraus wieder abzurufen. Die starken Möglichkeiten der MongoDB, von vielen Programmiersprachenaus angesprochen zu werden, unter anderem auch von C#, ist es für Entwickler einfach, mit Hilfe in der selben Programmiersprache geschriebenen Quellzeilen, in einem objekt-orientierten, Dokumenten-zentrierten Datenbank-Modell die Daten abzuspeichern und abzurufen. Hier könnten dann Diskussionen geführt werden, ob eine Mischung von relationalen und Dokumenten-basierten Datenbankmodellen eine gute Idee ist. Und wenn man sich für eine Vorgehensweise entschieden hat, entweder komplett relational oder aber die Mischung aus relationaler und Dokumenten-basierter Datenspeicherung, diese Erweiterung umzusetzen. Dazu würde auch die Erweiterung des REST APIs gehören, die die Auslieferung der erweiterten Detail-Daten mit zur Verfügung stellen muss. Bei einer rein relationalen

Datenspeicherung nur in der SQL-Datenbank müssten hier neue Tabellen mit einer entsprechenden Fremdschlüssel-Verknüpfung von der erfassten Maschine zu den Detail-Daten eingefügt werden. Auch die Programmierung müsste für dieses Szenario in geeigneter Art und Weise angepasst werden.

Diese beiden hier skizzierten Erweiterungen der Basis-App der InventarViewerApp sind dann ein größeres Unterfangen und Bewegen das Projekt von einem Proof-of-Concept mehr in eine Richtung, in der es schon in geringen Umfang produktiv eingesetzt werden könnte. Wie in allen solchen Projekten kommen nach einem Startschuss weitere Ideen und Anforderungen dazu, die das Projekt dann kontinuierlich wachsen lassen und dieses immer mehr erweitert wird und mehr und mehr einsatzfähiger wird. Hier sind dann aber die Grenzen der Auszubildenden erreicht und ab diesem Zeitpunkt müsste das PoC-Projekt in die Hände von ausgelernten und erfahrenen Anwendungsentwickler*innen, unter Führung eines/r Senior-Entwickler*in, gelegt werden, die dieses Projekt dann nach und nach in zur Produktionsreife weiterentwickeln.

Ausblick

Das Projekt ist auch dafür geeignet, den Ausbildungsgang Fachinformatiker*in Daten- und Prozessanalyse mitzuintegrieren, der voraussichtlich zukünftig ebenfalls bei der GWDG etabliert werden soll. Eventuell sind hierfür noch ein paar kleine Anpassungen/Ergänzungen dieses Projekts notwendig. Aber der Kern, was die Programmierung und die verwendeten Technologien und Programme angeht, sollte dann auch für diese drei Ausbildungsgänge in den ersten beiden gemeinsamen Lehrjahren passend und ausreichend sein.

Zusammenfassung

Das Projekt InventarWorkerService dient der gemeinsamen Ausbildung von Fachinformatiker*innen für Anwendungsentwicklung und Systemintegration bei der GWDG ab dem Ausbildungsjahr 2025. Es umfasst die Entwicklung eines plattformübergreifenden Systemdienstes und zugehöriger Anwendungen, die Hard- und Softwareinventarisierung automatisieren und die Zusammenarbeit der Auszubildenden fördern.

Gemeinsame Ausbildung ab 2025: Fachinformatiker*innen für Anwendungsentwicklung und Systemintegration werden in den ersten beiden Lehrjahren gemeinsam unterrichtet und arbeiten in Programmierübungen zusammen.

Projektumfang und Technologien: Das Projekt beinhaltet die Entwicklung eines Systemdienstes, Kommandozeilenprogramme, PowerShell-Cmdlets, Nutzung von SQL- und NoSQL-Datenbanken sowie REST APIs, programmiert in C# mit .Net Framework und JetBrains Rider als IDE.

Plattformübergreifende Ausführung: Die Anwendungen laufen auf Windows, macOS und Linux ohne Anpassungen, was durch eine gemeinsame Codebasis ermöglicht wird.

InventarWorkerService: Kernkomponente, die Hard- und Softwaredaten des Systems erfasst und über eine REST API bereitstellt, um von anderen Programmen abgefragt zu werden.

Zusätzliche Teilprojekte: Kommandozeilenprogramme zur Statusabfrage und Steuerung des Dienstes, inklusive grafischer ASCII-Steuerung und PowerShell-Cmdlet für plattformunabhängige Bedienung.

Eingesetzte Hilfsmittel: JetBrains Rider als plattformübergreifende IDE, NuGet-Pakete für erweiterte Funktionen, GitLab für Versionskontrolle, SQLite und MongoDB als Datenbanken sowie PSGallery für PowerShell-Module.

Erweiterungsmöglichkeiten: Ausbau der InventarViewerApp um Maschinenregistrierung, erweiterte Datenbankstrukturen, periodische Datensammlung und Nutzung von NoSQL-Datenbanken zur besseren Verwaltung großer JSON-Datenbestände.

Zukunftsperspektive: Möglichkeit, den Ausbildungsgang Fachinformatiker*in Daten- und Prozessanalyse einzubinden, sodass das Projekt die ersten beiden Lehrjahre mehrerer Fachrichtungen abdecken kann.

Summary

The InventarWorkerService project is designed for the joint training of IT specialists in Application Development and System Integration at GWDG, starting from the 2025 training year. It encompasses the development of a cross-platform system service and its associated applications, which automate hardware and software inventory and foster collaboration among trainees.

Joint Training from 2025: IT specialists for Application Development and System Integration will be taught together in the first two years of their apprenticeship and will collaborate on programming exercises.

Project Scope and Technologies: The project includes the development of a system service, command-line programs, PowerShell cmdlets, the use of SQL and NoSQL databases, and REST APIs, programmed in C# with the .NET Framework and JetBrains Rider as the IDE.

Cross-Platform Execution: The applications run on Windows, macOS, and Linux without modification, enabled by a common codebase.

InventarWorkerService: The core component that captures the system's hardware and software data and makes it available via a REST API to be queried by other programs.

Additional Sub-projects: Command-line programs for status inquiry and control of the service, including a graphical ASCII control interface and a PowerShell cmdlet for platform-independent operation.

Tools Used: JetBrains Rider as a cross-platform IDE, NuGet packages for extended functionality, GitLab for version control, SQLite and MongoDB as databases, and the PSGallery for PowerShell modules.

Expansion Possibilities: Extending the InventarViewerApp to include machine registration, expanded database structures, periodic data collection, and the use of NoSQL databases for better management of large JSON datasets.

Future Outlook: The potential to integrate the "IT Specialist for Data and Process Analysis" training path, allowing the project to cover the first two years of training for multiple specializations.

Anhang

Im Anhang habe ich zwei Kis mit dem folgenden Prompt „beschreibe bitte, was die Programme dieser Solution und Ihre Teilprojekte tun.“ gebeten, die Programme der Solution und deren Teilprojekte zu beschreiben, was diese tun.

Gemeni-cli

Gesamtzweck der Solution

Es handelt sich um ein System zur **Inventarisierung von Hardware- und Software-Informationen**

von Computern. Das System besteht aus mehreren Teilen:

1. Einem **Hintergrunddienst (Worker Service)**, der die Daten auf einem Rechner sammelt.
2. Einer **Desktop-Anwendung (Viewer App)**, um die gesammelten Daten von allen Rechnern anzuzeigen.
3. Mehreren **Steuerungskomponenten**, um den Dienst per Kommandozeile oder PowerShell zu verwalten.
4. Einem **Dokumentationsprojekt**, das die APIs und die Nutzung beschreibt.

Die Teilprojekte im Detail

1. Datensammlung (InventarWorker...)

* `**InventarWorkerService**` : Das ist das Herzstück der Datensammlung. Es ist ein **Hintergrunddienst**, der auf einem Zielrechner läuft.
  * Worker.cs: Enthält die Hauptlogik, die periodisch ausgeführt wird.
  * Services/HardwareInventoryService.cs: Sammelt detaillierte Informationen über die Hardware (CPU, RAM, Festplatten etc.).
  * Services/SoftwareInventoryService.cs: Sammelt Informationen über installierte Software und laufende Prozesse.

* Controllers/InventarController.cs: Stellt eine **API** bereit, über die der Dienst seine gesammelten Daten auf Anfrage (z.B. per HTTP) zur Verfügung stellt.

* `**InventarWorkerCommon**`: Eine Klassenbibliothek, die von anderen Projekten genutzt wird.

* Models/: Definiert die **Datenmodelle** für die gesammelten Informationen (z.B. CpuInfo, DiskInfo, SoftwareInfo). Dies stellt sicher, dass sowohl der Sammeldienst als auch die Anzeige-App dieselbe Datenstruktur verwenden.

2. Datenanzeige und -verwaltung (InventarViewer...)

* `**InventarViewerApp**`: Eine Anwendung mit einer textbasierten Benutzeroberfläche (TUI - Text-based User Interface), um die Inventardaten zentral anzuzeigen.

* UI/: Enthält die Logik für die verschiedenen Ansichten der Benutzeroberfläche (MainWindow, HardwareView, SoftwareView).

* Services/ApiService.cs: Kommuniziert mit der API des InventarWorkerService, um die Daten von den verschiedenen Rechnern abzurufen.

* Services/MongoDbService.cs & DatabaseService.cs: Deutet darauf hin, dass die abgerufenen Daten in einer eigenen **Datenbank** (vermutlich MongoDB) gespeichert werden, um eine zentrale Übersicht zu ermöglichen.

* Controllers/InventarController.cs: Stellt eine eigene, erweiterte API bereit, um z.B. auf die aggregierten Daten in der Datenbank zuzugreifen.

3. Steuerung und Automatisierung (CtrlWorker...)

Diese Projekte dienen dazu, den InventarWorkerService auf den Rechnern zu steuern (starten,

stoppen, Status abfragen).

* `**CtrlWorkerServiceCmdlet**`: Erstellt ein **PowerShell-Cmdlet**

(InvokeWorkerServiceControlCmdlet.cs). Damit kann der Dienst direkt und "nativ" in

PowerShell-Skripten verwaltet werden.

* `**CtrlWorkerServicePS**`: Enthält fertige **PowerShell-Skripte** (.ps1), die das Cmdlet verwenden,

um die Verwaltung zu vereinfachen.

* `**CtrlWorkerServiceApp**`: Eine einfache **Kommandozeilen-Anwendung**, die wahrscheinlich dieselbe

Steuerungslogik wie das PowerShell-Cmdlet nutzt, aber ohne PowerShell auskommt.

* `**CtrlWorkerCommon**`: Eine gemeinsame Bibliothek für die Steuerungslogik, die von der Cmdlet-

und der Kommandozeilen-App genutzt wird.

4. Hilfsprogramme und Sonstiges

* `**ServiceStatusReaderApp**`: Ein kleines, spezialisiertes Konsolenprogramm, das den Status des

Worker-Dienstes ausliest. Wahrscheinlich greift es auf eine Status-Datei zu, die der Dienst

schreibt (FileBasedStatusService.cs im Worker-Projekt ist ein starker Hinweis darauf).

* `**main/**`: Enthält **SQL-Skripte**. Diese definieren die Tabellen (Machines, HardwareInventories,

SoftwareInventories) für eine relationale Datenbank (z.B. SQL Server oder PostgreSQL), die

alternativ oder zusätzlich zu MongoDB genutzt werden könnte.

* `**Writerside/**` : Ein **Dokumentationsprojekt** (vermutlich mit JetBrains Writerside erstellt).

* topics/: Enthält die eigentlichen Dokumentationsartikel im Markdown-Format.

* specifications/: Beinhaltet swagger.json-Dateien, was bedeutet, dass die

API-Dokumentation automatisch aus dem Code generiert und hier eingebunden wird.

Zusammenfassend ist dies eine gut strukturierte Solution für ein klassisches

Client-Server-Inventarisierungssystem mit klaren Trennungen zwischen Datenerfassung,

Datenanzeige, Steuerung und Dokumentation.

Claude-Code

Die **InventarWorkerService Solution** ist ein .NET 9 basiertes System zur Erfassung und Verwaltung von

Hardware- und Software-Inventaren auf verschiedenen Betriebssystemen (Windows, macOS, Linux).

Hauptkomponenten:

🔧 **InventarWorkerService (Kern-Service)**

- **Zweck**: Plattformübergreifender Background-Service zur kontinuierlichen Inventarerfassung
- **Features**: REST API mit Swagger-Dokumentation, Hardware-/Software-Inventarisierung
- **Endpunkte**: Status, Hardware-Info, Software-Info, vollständiges Inventar

📊 **InventarViewerApp (Viewer/Dashboard)**

- **Zweck**: TUI-basierte Anwendung zum Anzeigen der Inventardaten
- **Modi**: Terminal UI (Standard) oder Web API Modus (--api Parameter)
- **Datenbankanbindung**: SQLite und MongoDB Unterstützung

🎮 **ServiceStatusReaderApp**

- **Zweck**: Monitoring-Tool für Service-Status und erweiterte Überwachung
- **Features**: Service-Pfad-Management, erweiterte Monitoring-Funktionen

🛠️ **Control-Tools**

- **CtrlWorkerServiceApp**: Konsolen-Steuerungsanwendung für den Worker Service
- **CtrlWorkerServiceCmdlet**: PowerShell Cmdlet zur Service-Kontrolle
- **CtrlWorkerServicePS**: PowerShell-Skripte für Service-Management

📚 Gemeinsame Bibliotheken

- **InventarWorkerCommon**: Datenmodelle für Hardware/Software-Inventar
- **CtrlWorkerCommon**: Plattformübergreifende Service-Controller

Das System ermöglicht automatische Inventarisierung von Hardware (CPU, RAM, Festplatten, Netzwerk)

und Software (Prozesse, Services, Runtime-Informationen) mit REST API-Zugriff und verschiedenen

Verwaltungstools.